A- CONSULTAS SIMPLES

1)Obtener una lista con los nombres de las distintas oficinas de la empresa.

```
SELECT descripción

FROM oficinas;
```

2) Obtener una lista de todos los productos indicando descripción del producto, su precio de costo y su precio de costo IVA incluído (tomar el IVA como 21%).

```
SELECT descripcion, precio AS precio_costo, precio * 1.21 AS precio_iva

FROM productos;
```

3) Obtener una lista indicando para cada empleado apellido, nombre, fecha de cumpleaños y edad.

```
SELECT apellido, nombre, fecha_nacimiento , TIMESTAMPDIFF (YEAAR, fecha_nacimiento, CURDATE()) AS edad

FROM empleados;
```

4) Listar todos los empleados que tiene un jefe asignado.

```
SELECT *

FROM empleados

WHERE cod_jefe IS NOT NULL;
```

5) Listar los empleados de nombre “María” ordenado por apellido.

```
SELECT *

FROM empleados

WHERE nombre= ‘María’

ORDER BY apellido;
```

6) Listar los clientes cuya razón social comience con “L” ordenado por código de cliente.

```
SELECT *

FROM clientes

WHERE razón_social LIKE ‘L%’

ORDER BY cod_cliente;
```

7) Listar toda la información de los pedidos de Marzo ordenado por fecha de pedido.

```
SELECT *

FROM pedidos

WHERE MONTH (fecha_pedido)= 3
```

```
ORDER BY fecha_pedido;
```

8) Listar las oficinas que no tienen asignado director.

```
SELECT *
FROM oficinas
WHERE codigo_director IS NULL;
```

9) Listar los 4 productos de menor precio de costo.

```
SELECT *
FROM productos
ORDER BY precio ASC
LIMIT 4;
```

10) Listar los códigos de empleados de los tres que tengan la mayor cuota.

```
SELECT cod_empleado
FROM datos_contratos
ORDER BY cuota DESC
LIMIT 3;
```

CONSULTAS MULTITABLAS

1) De cada producto listar descripción, razón social del fabricante y stock ordenado por razón social y descripción.

```
SELECT p.descripcion, f.razon_social, p.cantidad_stock
FROM productos pr
JOIN fabricantes f ON pr.cod_fabricante=f.cod_fabricante
ORDER BY f.razon_social, pr.descripcion;
```

2) De cada pedido listar código de pedido, fecha de pedido, apellido del empleado y razón social del cliente.

```
SELECT pe.cod_pedido, pe.fecha_pedido, e.apellido, c.razon_social
FROM pedidos pe
JOIN empleados e ON pe.cod_empleado=e.cod_empleado
JOIN clientes c ON pe.cod_cliente=c.cod_cliente;
```

3) Listar por cada empleado apellido, cuota asignada, oficina a la que pertenece ordenado en forma descendente por cuota.

```
SELECT e.apellido, d.cuota, o.descripcion
```

```
FROM empleados e
JOIN datos_contratos d ON e.cod_empleado=d.cod_empleado
JOIN oficinas o ON e.cod_oficina=o.cod_oficina
ORDER BY d.cuota DESC;
```

4) Listar sin repetir la razón social de todos aquellos clientes que hicieron pedidos en Abril.

```
SELECT DISTINCT c.razon_social
FROM clientes c
JOIN pedidos pe ON c.cod_cliente=pe.cod_cliente
WHERE MONTH(pe.fecha_pedido)=4;
```

5) Listar sin repetir los productos que fueron pedidos en Marzo.

```
SELECT DISTINCT pr.*
FROM productos pr
JOIN detalle_pedidos dp ON pr.cod_producto=dp.cod_producto
JOIN pedidos pe ON dp.cod_pedido=pe.cod_pedido
WHERE MONTH (pe.fecha_pedido)=3;
```

6) Listar aquellos empleados que están contratados por más de 10 años ordenado por cantidad de años en forma descendente.

```
SELECT e.apellido, e.nombre, TIMESTAMPDIFF(YEAR, d.fecha_contrato, CURDATE()) AS anios
FROM empleados e
JOIN datos_contratos d ON e.cod_empleado=d.cod_empleado
WHERE TIMESTAMPDIFF (YEAR, d.fecha_contrato, CURDATE()) AS anios
ORDER BY años DESC;
```

7) Obtener una lista de los clientes mayoristas ordenada por razón social.

```
SELECT c.*
FROM clientes c
JOIN listas l ON c.cod_lista=l.cod_lista
WHERE l.descripcion= ‘mayorista’
ORDER BY c.razon_social;
```

8) Obtener una lista sin repetir que indique qué productos compró cada cliente, ordenada por razón social y descripción.

```
SELECT DISTINCT c.razon_social, pr.descripcion
FROM clients c
JOIN pedidos pe ON c.cod_cliente=pe.cod_cliente
JOIN detalle_pedidos dp ON pe.cod_pedido=dp.cod_pedido
JOIN productos pr ON dp.cod_producto=pr.cod_producto
ORDER BY c.razon_social, pr.descripcion;
```

9) Obtener una lista con la descripción de aquellos productos cuyo stock está por debajo del punto de reposición indicando cantidad a comprar y razón social del fabricante ordenada por razón social y descripción.

```
SELECT pr.descripcion, (pr.punto_reposicion – pr.cantidad_stock) AS cantidad_a_comprar, f.razon_social
FROM productos pr
JOIN fabricantes f ON pr.cod_fabricante=f.cod_fabricante
WHERE pr.cantidad_stock < pr.punto_reposicion
ORDER BY f.razon_social, pr.descripcion;
```

10) Listar aquellos empleados cuya cuota es menor a 50000 o mayor a 100000.

```
SELECT e.*, d.cuota
FROM empleados e
JOIN datos_contratos d ON e.cod_empleado=d.cod_empleado
WHERE d.cuota < 50000 OR d.cuota > 100000;
```

B- TABLAS CLIENTES, PRODUCTOS, ITEM_VENTAS, VENTAS

1) Obtener la cantidad de unidades máxima.

```
SELECT MAX (cantidad) AS cantidad_maxima
FROM item_ventas;
```

2) Obtener la cantidad total de unidades vendidas del producto c.

```
SELECT SUM (cantidad) AS total_unidades
FROM item_ventas
WHERE codigo_producto= ‘c’;
```

3) Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado de mayor a menor por las cantidades vendidas.

SELECT pr.nombre_producto, SUM (iv.cantidad) AS total_vendido

FROM productos pr

JOIN item_ventas iv ON pr.codigo_producto=iv.codigo_producto

GROUP BY pr.codigo_producto, pr.nombre_producto

GROUP BY total_vendido DESC;

4) Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado alfabéticamente por nombre de producto para los productos que vendieron mas de 30 unidades.

SELECT pr.nombre_producto, SUM (iv.cantidad) AS total_vendido

FROM productos pr

JOIN item_ventas iv ON pr.codigo_producto=iv.codigo_producto

GROUP BY pr.codigo_producto, pr.nombre_producto

HAVING SUM (iv.cantidad) > 30

ORDER BY pr.nombre_producto ASC;

5) Obtener cuantas compras (1 factura = 1 compra) realizó cada cliente indicando el código y nombre del cliente ordenado de mayor a menor.

SELECT c.codigo_cliente, c.nombre, COUNT (v.numero_factura) AS cantidad_compras

FROM clientes c

JOIN ventas v ON c.codigo_cliente = v.codigo_cliente

GROUP BY c.codigo_cliente, c.nombre

ORDER BY cantidad_compras DESC;

6) Promedio de unidades vendidas por producto, indicando el código del producto para el cliente 1

SELECT iv.codigo_producto, AVG(iv.cantidad) AS promedio_unidades

FROM item_ventas iv

JOIN ventas v ON iv.numero_factura = v.numero_factura

WHERE v.codigo_cliente=1

GROUP BY iv.codigo_producto;